Guía práctica de Git y GitHub

Manual reorganizado de principio a fin · Git Bash, GitHub CLI (gh), Lazygit, ramas, versiones y recuperación · Windows · Octubre de 2026.

Objetivo: comenzar con una carpeta existente y crear, vincular y publicar un repositorio local y remoto en una sola secuencia. Después, aprender los casos alternativos y las herramientas de mantenimiento.

Convenciones: los comandos se ejecutan en Git Bash; sustituí TU_USUARIO, MiProyecto, HASH y las rutas por los datos reales. Las líneas precedidas por # son comentarios explicativos, no pasos obligatorios.

Índice

1. Inicio rápido: carpeta + Git + GitHub con gh
2. Instalación, identidad y autenticación
3. Los cuatro escenarios iniciales
4. Comprobar la carpeta, el remoto y la rama
5. Trabajo cotidiano: add, commit, push, fetch y pull
6. Ramas y fusión; main y master
7. Historial, tags y versiones de PLANIALTIS
8. Recuperación segura, restore, revert, reset y worktree
9. Lazygit: interfaz y lectura del estado
10. GitHub CLI: crear, listar, consultar y eliminar
11. Archivos ignorados y proyectos LaTeX / DWG
12. Trabajar en dos computadoras y colaborar
13. Diagnóstico, errores y referencia de comandos

1. Inicio rápido: carpeta + Git + GitHub con gh

Este es el procedimiento principal. Partís de una carpeta con archivos y querés crear el repositorio Git local, crear su contraparte en GitHub, vincularlos y subir la primera versión sin abrir el navegador.

Caso recomendado: repositorio privado

```
cd /c/GITHUB/MiProyecto
git init -b main
git status
# Crear o revisar .gitignore antes de agregar archivos
git add .
git commit -m "Version inicial"
gh auth login
gh repo create MiProyecto --private --source=. --remote=origin --push
```

Si ya estás autenticado con gh, omití gh auth login. gh repo create crea el repositorio remoto, añade origin y publica la rama con --push. No ejecutes además git remote add origin para el mismo remoto.

La misma operación para un repositorio público

```
gh repo create MiProyecto --public --source=. --remote=origin --push
```

Verificar el resultado

```
git rev-parse --show-toplevel
git remote -v
git branch -vv
git status -sb
gh repo view --web
```

El primer comando identifica la raíz local; remote -v muestra la URL de descarga y publicación; branch -vv muestra el seguimiento de la rama. gh repo view --web abre el repositorio en GitHub.

Importante: Si la carpeta ya tenía Git o un remoto origin, comprobá primero git status y git remote -v. No repitas a ciegas los comandos de inicialización.

2. Instalación, identidad y autenticación

Instalar y comprobar

Instalá Git for Windows desde git-scm.com/downloads/win y GitHub CLI desde cli.github.com. Podés utilizar winget si está disponible:

```
winget install --id Git.Git -e
winget install --id GitHub.cli -e
git --version
gh --version
```

Identidad de autor: una vez por computadora

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
git config --global --list
```

Estos datos se guardan en los commits. No son las credenciales de acceso a GitHub. Podés cambiar la identidad solo para un repositorio usando git config --local.

Autenticarse con GitHub CLI

```
gh auth login
gh auth status
```

Elegí GitHub.com y el protocolo HTTPS si es tu forma de trabajo. El asistente ofrece autenticación en el navegador. Git Credential Manager puede administrar por separado las credenciales que utiliza Git para HTTPS.

3. Los cuatro escenarios iniciales

A. Carpeta existente y repositorio nuevo en GitHub: todo con gh

```
cd /c/GITHUB/MiProyecto
git init -b main
git add .
git commit -m "Version inicial"
gh repo create MiProyecto --private --source=. --remote=origin --push
```

B. Carpeta existente y repositorio vacío ya creado en GitHub

Si el repositorio remoto está vacío (sin README ni commits), no necesitás gh repo create:

```
cd /c/GITHUB/MiProyecto
git init -b main
git add .
git commit -m "Version inicial"
git remote add origin https://github.com/TU_USUARIO/MiProyecto.git
git push -u origin main
```

remote add origin guarda la URL; push -u publica y configura el seguimiento. Si el remoto ya tiene commits, primero hay que revisar cómo integrarlos; no fuerces la publicación.

C. El repositorio ya tiene archivos en GitHub: clonar

```
cd /c/GITHUB
git clone https://github.com/TU_USUARIO/MiProyecto.git
cd MiProyecto
git status
git remote -v
```

Clonar descarga el proyecto, su historial y la configuración de origin. No hagas git init ni remote add después del clone.

D. Solo Git local, sin GitHub

```
cd /c/GITHUB/MiProyecto
git init -b main
git add .
git commit -m "Version inicial"
git log --oneline
```

Los commits se guardan en la carpeta oculta .git. Podés agregar un remoto más adelante. Si el proyecto ya es un repositorio, no hace falta reinicializarlo.

4. Comprobar la carpeta, el remoto y la rama

```
pwd
git rev-parse --show-toplevel
git status -sb
git remote -v
git branch -vv
git config --show-origin --get remote.origin.url
```

origin es un alias convencional, no el nombre de una rama. main y master son nombres posibles de la rama principal. No hay diferencia funcional intrínseca. git branch --show-current informa cuál está activa.

Si origin apunta al lugar equivocado

```
git remote set-url origin https://github.com/TU_USUARIO/MiProyecto.git
git remote -v
```

Importante: Cambiar origin no mueve archivos ni cambia el historial local; sí cambia el destino de operaciones futuras con ese remoto.

5. Trabajo cotidiano: add, commit, push, fetch y pull

Al comenzar una sesión

```
git status
git fetch origin
git status -sb
git pull
```

Fetch actualiza la información local sobre el remoto, sin integrar cambios a tus archivos. Pull obtiene e integra cambios; revisá primero si hay modificaciones locales o divergencias.

Después de editar y guardar

```
git status
git diff
git add .
git diff --staged --stat
git commit -m "Describir el cambio"
git push
```

add prepara una instantánea para el próximo commit; commit registra localmente; push publica los commits pendientes. git add . actúa desde la carpeta actual hacia abajo; git add -A prepara todos los cambios del repositorio.

Primer push de una rama sin seguimiento

```
git push -u origin main
```

La opción -u equivale a --set-upstream: configura el seguimiento de main respecto de origin/main. Si la rama se llama master, usá master.

Cuántos commits faltan subir

```
git fetch origin
git status -sb
git log origin/main..main --oneline
git log main..origin/main --oneline
```

El primer log enumera commits solo locales; el segundo, commits solo remotos conocidos. ahead 3 indica tres commits locales pendientes. behind indica que hay commits remotos por incorporar.

Preparar solo algunos archivos

```
git add secciones/cap-1.tex
git add bibliografia.bib
git diff --staged
git commit -m "Actualizar capitulo y referencias"
```

6. Ramas y fusión; main y master

Crear una rama experimental

```
git status
git switch main
git switch -c prueba-correccion
# Editar archivos
git add .
git commit -m "Probar correccion"
```

Una rama es un nombre móvil que apunta a un commit; no es una carpeta separada. El asterisco en git branch identifica la rama activa.

Volver a la rama principal y fusionar

```
git switch main
git merge prueba-correccion
git push
git branch -d prueba-correccion
```

Merge incorpora el historial de la rama experimental. Si hubo cambios incompatibles, puede haber conflictos que debés resolver antes de completar la fusión.

Ramas remotas y seguimiento

```
git branch
git branch -a
git branch -vv
git fetch origin
```

Las referencias origin/main son datos locales sobre el estado conocido del remoto. Se actualizan con fetch; no se consultan automáticamente en cada git status.

7. Historial, tags y versiones de PLANIALTIS

Consultar el historial

```
git log --oneline --graph --all --decorate
git log -10 --oneline
git show HASH --stat
git diff HASH1 HASH2
```

Cada commit tiene un hash. Un tag señala un hito, como una versión validada de PLANIALTIS; no hace falta etiquetar cada commit.

Marcar y publicar una versión

```
git tag -a V4.80 -m "PLANIALTIS V4.80 validada"
git push origin V4.80
git tag --list "V4.*"
```

Los tags no se envían necesariamente con un git push normal. Un tag existente no debería reutilizarse para otra versión publicada.

Etiquetar un commit anterior

```
git log --oneline
git tag V4.79 HASH
git push origin V4.79
```

Para proyectos LaTeX, podés etiquetar entregas como V1.00, V1.01, etc. En archivos DWG, Git conserva versiones, pero no compara gráficamente geometrías.

8. Recuperación segura, restore, revert, reset y worktree

Ver una versión antigua en otra carpeta, sin alterar main

```
git worktree add --detach ../MiProyecto_ANTERIOR HASH
git worktree list
# Al terminar, cerrar archivos abiertos
git worktree remove ../MiProyecto_ANTERIOR
```

Es la opción más cómoda para comparar versiones. El worktree comparte el historial, pero tiene su propia carpeta de archivos.

Recuperar un archivo de un commit anterior

```
git status
git restore --source=HASH -- ruta/archivo.tex
git diff -- ruta/archivo.tex
```

Importante: Restore puede sobrescribir modificaciones no guardadas en el archivo afectado. Verificá primero git status y hacé respaldo si corresponde.

Deshacer un commit sin borrar historial

```
git revert HASH
git push
```

Revert crea un nuevo commit con el cambio inverso. Es preferible para deshacer commits ya publicados cuando no querés reescribir la historia.

Restaurar todo el contenido de una versión antigua y registrarlo

```
git status
git restore --source=HASH --staged --worktree :/
git diff --staged --stat
git commit -m "Restaurar contenido de version anterior"
```

Conserva el historial, pero modifica archivos del proyecto. git reset --hard HASH mueve la rama y descarta cambios locales rastreados: es una operación de riesgo y no debe usarse como primera opción.

Guardar temporalmente modificaciones

```
git stash push -u -m "Trabajo temporal"
git stash list
git stash pop
```

Stash no sustituye al commit ni al respaldo. Aplicarlo puede producir conflictos.

9. Lazygit: interfaz y lectura del estado

Lazygit es una interfaz de terminal para Git. No es lazy.nvim, que administra complementos de Neovim. Desde Git Bash, entrá al repositorio y ejecutá lazygit.

```
cd /c/GITHUB/MiProyecto
lazygit
```

En la interfaz podés revisar archivos modificados, preparar cambios, crear commits, consultar el historial, ramas y remotos. Los indicadores de ahead/behind dependen de las referencias remotas conocidas; fetch las actualiza.

Si aparece una flecha hacia arriba con 3 en main, normalmente significa tres commits locales por delante de su upstream conocido. Los commits de una rama se identifican por la historia de la referencia; un commit puede pertenecer al historial de varias ramas. Para salir, normalmente se utiliza q; Esc suele cerrar cuadros o volver al panel anterior.

Importante: Antes de operaciones destructivas en Lazygit, comprobá el repositorio, la rama y el estado. No confundas fetch (consultar/descargar referencias) con pull (integrar).

10. GitHub CLI: crear, listar, consultar y eliminar

Iniciar sesión y comprobar

```
gh auth login
gh auth status
gh repo list TU_USUARIO
```

Crear un repositorio y subir el proyecto actual

```
cd /c/GITHUB/MiProyecto
git init -b main
git add .
git commit -m "Version inicial"
gh repo create MiProyecto --private --source=. --remote=origin --push
```

Usá --public en lugar de --private para hacerlo público. Si el remoto ya existe, no uses este comando de creación: configurá la URL con git remote add origin o git remote set-url, según corresponda.

Consultar y abrir

```
gh repo view
gh repo view --web
gh repo list TU_USUARIO
```

Eliminar un repositorio remoto de GitHub

```
gh repo delete TU_USUARIO/MiProyecto
# Desde la carpeta de un repositorio vinculado:
gh repo delete
# Omitir la confirmacion interactiva (peligroso):
gh repo delete TU_USUARIO/MiProyecto --yes
```

Eliminar un repositorio de GitHub no elimina automáticamente la carpeta local ni su historial .git. A la inversa, borrar la carpeta local no elimina el remoto. Confirmá siempre propietario y nombre antes de eliminar.

Si faltan permisos para borrar

```
gh auth refresh -s delete_repo
gh auth status
```

La cuenta debe tener permisos suficientes sobre el repositorio. La ampliación del alcance de autenticación no concede por sí sola permisos administrativos que la cuenta no posea.

11. Archivos ignorados y proyectos LaTeX / DWG

Crear .gitignore para un proyecto LaTeX

```
touch .gitignore
notepad .gitignore
```

Ejemplo de contenido del archivo (una regla por línea):

```
*.aux
*.log
*.out
*.toc
*.lof
*.lot
*.synctex.gz
*.fls
*.fdb_latexmk
*.xdv
*.bcf
*.run.xml
*.blg
```

Los .tex, .bib y figuras suelen versionarse. Decidí explícitamente si querés incluir el PDF final. Para un proyecto con bibliografía, comprobá si los .bbl son reproducibles antes de ignorarlos.

Ignorar carpetas y archivos temporales de AutoCAD

```
/temporales/
*.bak
*.sv$
*.dwl
*.dwl2
```

Las reglas de .gitignore no eliminan del seguimiento archivos que Git ya registra. Para retirar una carpeta del índice sin borrarla físicamente:

```
git rm -r --cached temporales/
git add .gitignore
git commit -m "Ignorar archivos temporales"
```

Git conserva versiones de DWG, pero git diff no compara su geometría; para eso usá herramientas de comparación de AutoCAD. Evitá subir datos confidenciales de clientes sin autorización.

12. Trabajar en dos computadoras y colaborar

En la segunda computadora, clonar una sola vez

```
cd /c/GITHUB
git clone https://github.com/TU_USUARIO/MiProyecto.git
cd MiProyecto
git status
```

Rutina entre computadoras

En la computadora A: guardar, add, commit y push. En la computadora B: antes de editar, revisar status y pull. Repetir a la inversa al cambiar de equipo. Evitá editar el mismo archivo en dos equipos sin sincronizar.

Repositorios privados compartidos

El propietario invita a la otra persona desde GitHub, en la configuración de acceso o colaboradores. La persona invitada acepta y clona con su propia cuenta. Autor del commit, propietario del repositorio y cuenta autenticada son conceptos distintos.

Importante: No uses OneDrive para sincronizar simultáneamente una misma carpeta .git activa en dos equipos. Preferí un clon local por equipo y GitHub para intercambiar commits.

13. Diagnóstico, errores y referencia de comandos

Cinco consultas antes de modificar algo

```
git rev-parse --show-toplevel
git status -sb
git remote -v
git branch -vv
git log -5 --oneline --decorate
```

Mensajes frecuentes

fatal: not a git repository: no estás dentro de un repositorio; comprobá pwd y cd. remote origin already exists: consultá git remote -v y, si corresponde, usá git remote set-url. nothing to commit: no hay cambios preparados para registrar. ahead 3: tres commits locales pendientes según la referencia remota conocida. non-fast-forward: el remoto tiene una historia que debés revisar e integrar antes de publicar; no recurras automáticamente a --force.

Opciones frecuentes

Comando / opción	Significado
git commit -m	--message: mensaje del commit
git push -u	--set-upstream: seguimiento remoto
git status -sb	-s: short; -b: branch
git remote -v	-v: verbose; muestra URL
git branch -vv	detalle de commits y upstream
git switch -c	--create: crear y cambiar de rama
git branch -d	borrar rama local ya integrada
git add -A	preparar altas, cambios y eliminaciones
git diff --staged	comparar lo preparado
git log --oneline	un commit por línea
git restore --source	recuperar desde un commit o tag

Resumen: qué comando usar

Iniciar local: git init -b main. Crear GitHub y publicar: gh repo create ... --source=. --remote=origin --push. Vincular remoto existente: git remote add origin URL. Clonar: git clone URL. Publicar por primera vez: git push -u origin main. Uso diario: git status → git pull → editar → git add → git commit → git push.

Esta edición integra los procedimientos por función en lugar de conservar anexos acumulativos. Para comandos que cambian archivos o reescriben historia, revisá el estado y conservá un respaldo antes de ejecutarlos.